```
#include <stdio.h>
#include <stdint.h>
#include <string.h>

// ===== 64bit μ-op 구조체 =====
typedef struct {
    uint64_t raw_inst;      // 64bit 명령어
    char opcode[8];         // 예: "ADD", "LD", "MUL"
    int rd, rs1, rs2;
    int latency;            // 예상 지연 (cycles)
} riscv64_uop;

// ===== 디코딩 (간단 매핑) =====
void decode_uop(uint64_t raw, riscv64_uop* out) {
    out->raw_inst = raw;
    out->rd  = (raw >> 7) & 0x1F;
    out->rs1 = (raw >> 15) & 0x1F;
    out->rs2 = (raw >> 20) & 0x1F;

    uint8_t opcode = raw & 0x7F;
    if (opcode == 0x03) strcpy(out->opcode, "LD");
    else if (opcode == 0x33) strcpy(out->opcode, "ADD");
    else if (opcode == 0x3B) strcpy(out->opcode, "MUL");
    else strcpy(out->opcode, "NOP");

    out->latency = (strcmp(out->opcode, "MUL") == 0) ? 3 :
                   (strcmp(out->opcode, "LD") == 0) ? 2 : 1;
}

// ===== IPC 파이프라인 흐름 =====
void run_ipc_pipeline(riscv64_uop* uop) {
    printf("[Fetch] 0x%016llX\n", uop->raw_inst);
    printf("[Decode] %s r%d ← r%d, r%d\n", uop->opcode, uop->rd, uop->rs1, uop->rs2);
    printf("[Execute] 예상 지연: %d cycles\n", uop->latency);
    printf("[Writeback] r%d ← 결과 반영\n", uop->rd);
    printf("[Commit] 순서대로 확정됨\n\n");
}

// ===== 테스트 =====
int main() {
    uint64_t raw_insts[] = {
        0x0000000000010003, // LD r1 ← [r0]
        0x002081B3,         // ADD r3 ← r1, r2
        0x0051C3B3,         // MUL r7 ← r3, r5
        0x0062D433          // SUB r8 ← r4, r6 (treated as ADD here)
    };

    for (int i = 0; i < 4; i++) {
```

```
        riscv64_uop uop;
        decode_uop(raw_insts[i], &uop);
        run_ipc_pipeline(&uop);
    }

    return 0;
}
```